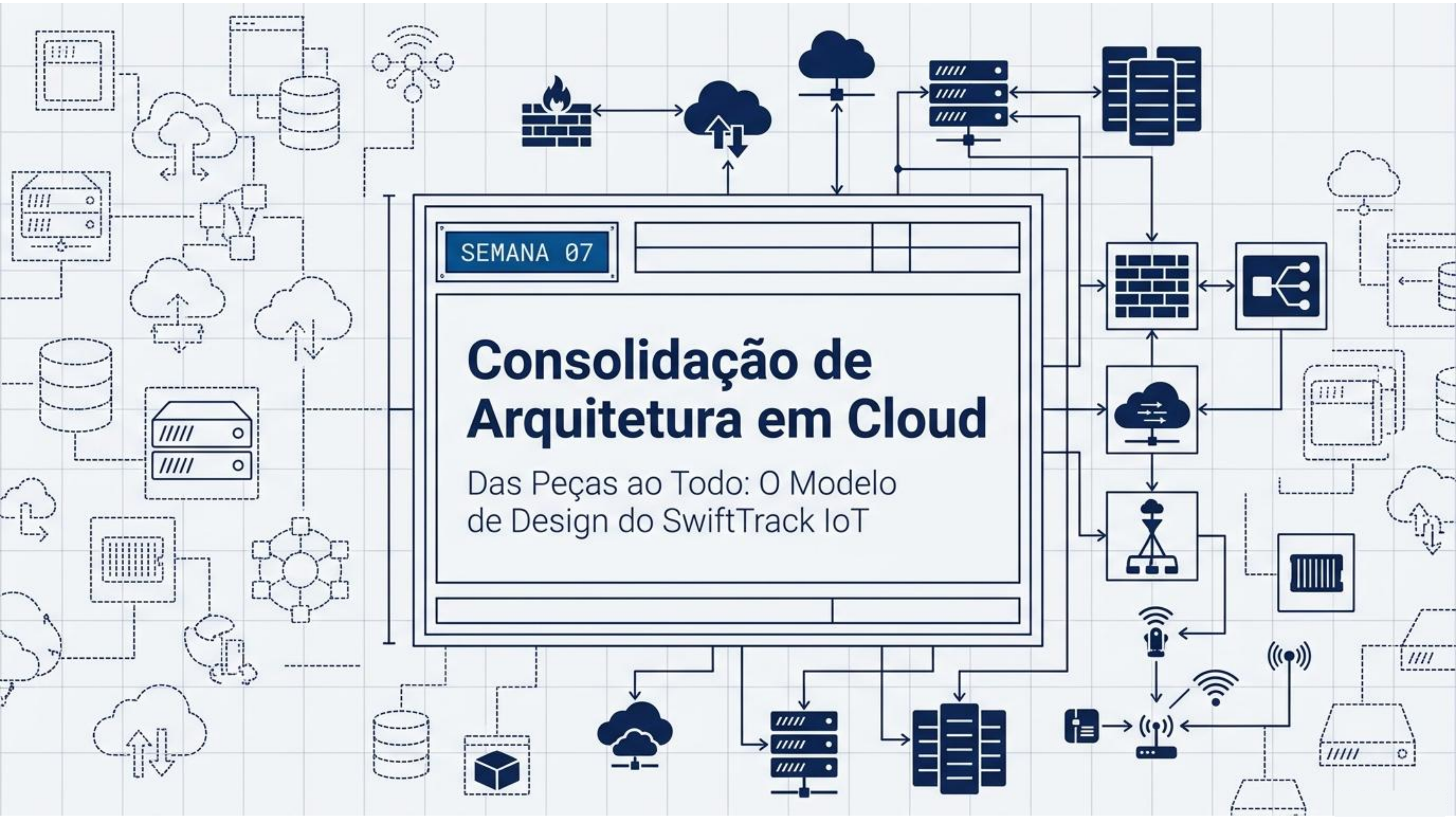


SEMANA 07

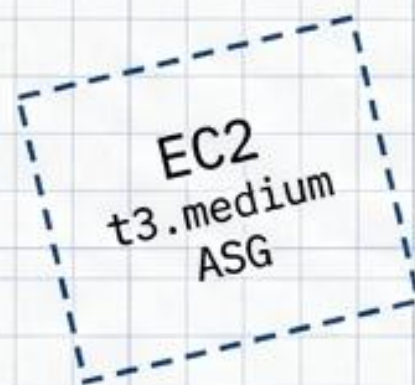
Consolidação de Arquitetura em Cloud

Das Peças ao Todo: O Modelo
de Design do SwiftTrack IoT

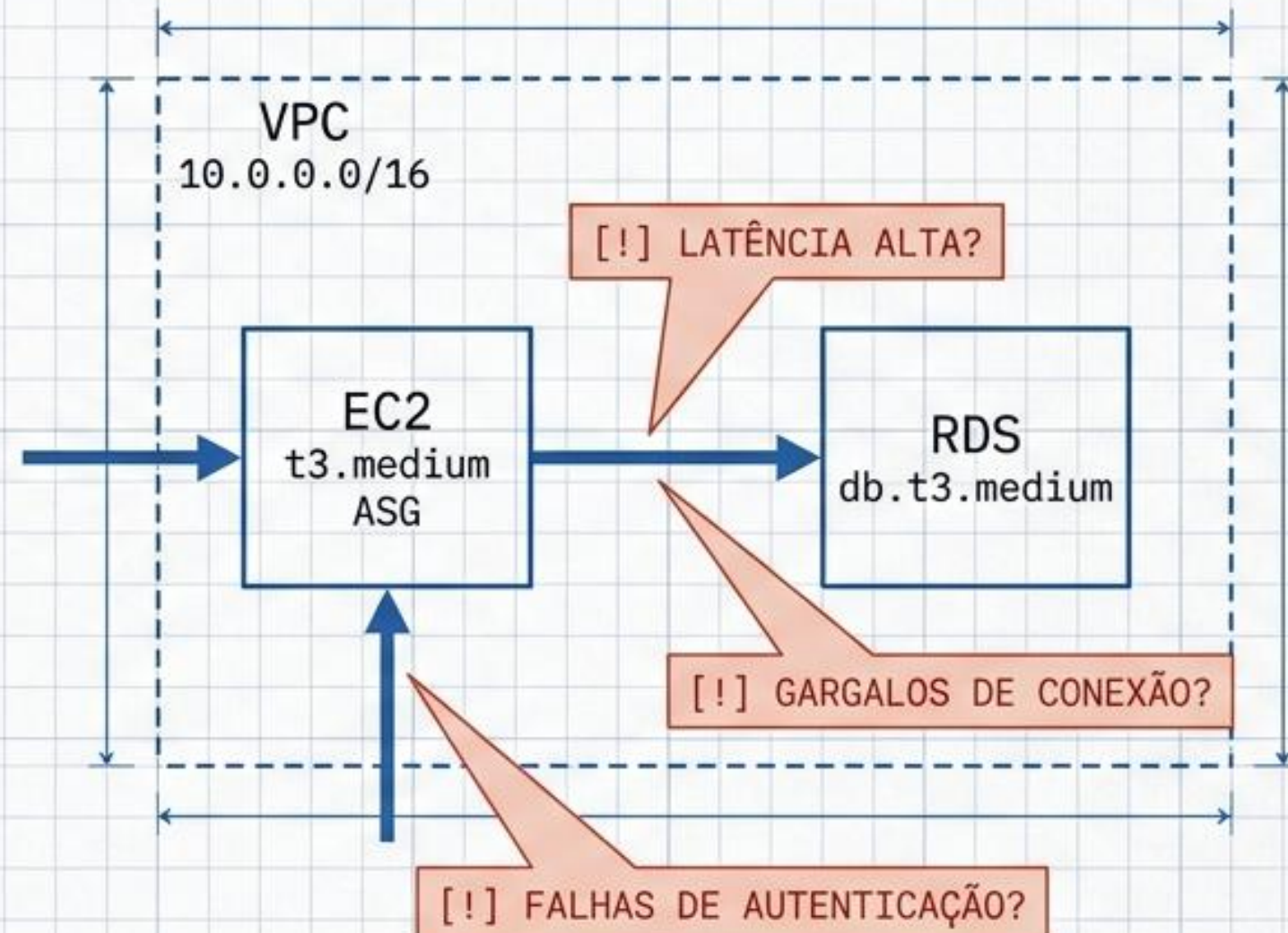


0 DESAFIO: CONECTANDO A INFRAESTRUTURA

SEMANAS 1-6



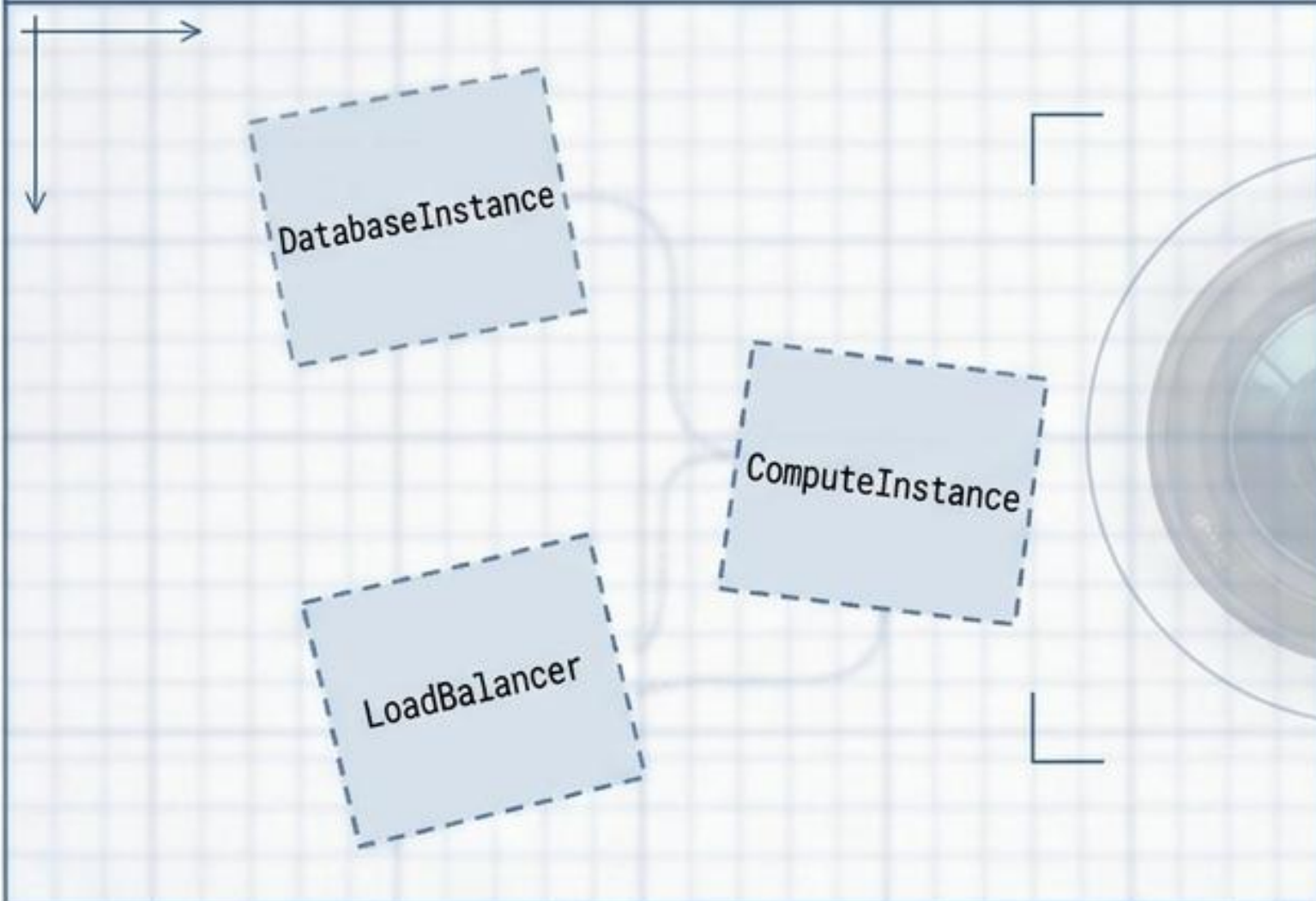
SEMANA 7: O MODELO DE DESIGN



O DESIGN DE COMPONENTES INDIVIDUAIS NÃO GARANTE UM SISTEMA FUNCIONAL. PRECISAMOS MAPEAR O CAMINHO EXATO (INPUT -> CAIXA PRETA -> OUTPUT) PARA VALIDAR A ARQUITETURA ANTES DA IMPLEMENTAÇÃO.

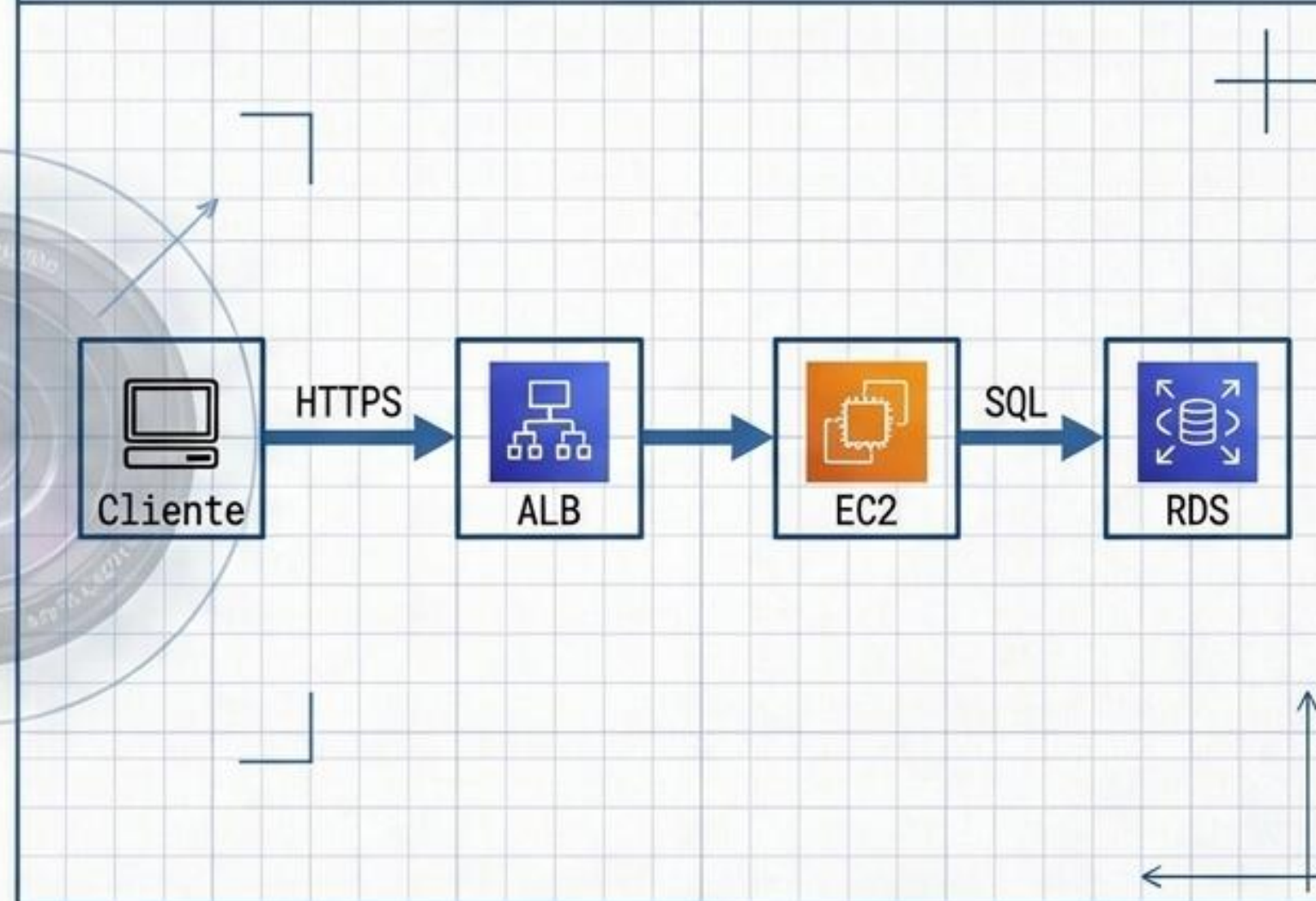
O PARADIGMA: ANÁLISE VS. DESIGN

MODELO DE ANÁLISE



Foco:	O QUÊ (Conceitual)
Abstração:	Alta (Pacotes, Classes de Análise)
Exemplo:	DatabaseInstance

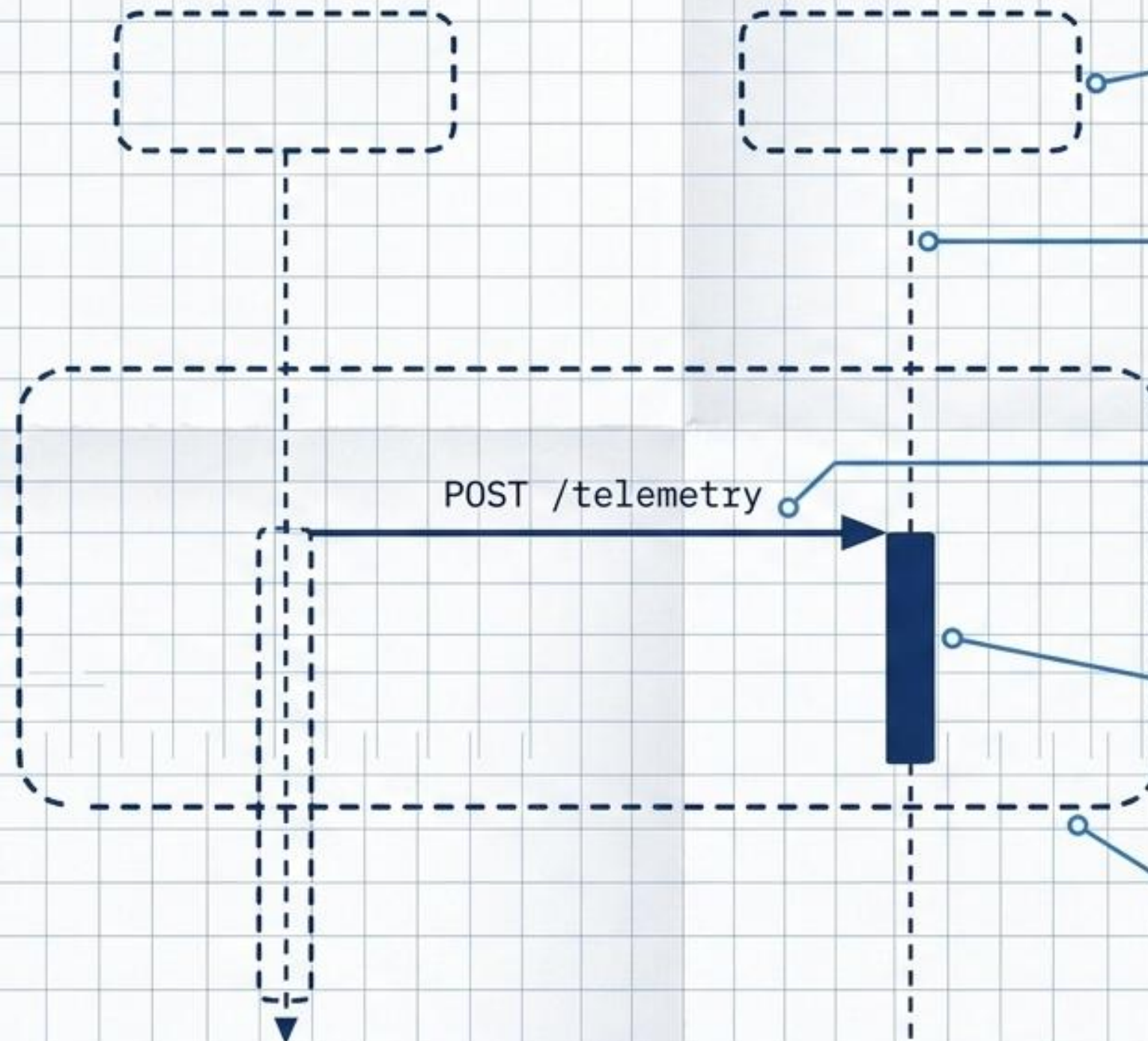
MODELO DE DESIGN



Foco:	COMO (Implementação Física)
Abstração:	Exata (Componentes AWS, Interfaces, Portas)
Exemplo:	Cliente -> ALB -> EC2 -> RDS

Lendo a Planta Baixa: A Anatomia da Sequência

Tempo (T)



Ator: Quem inicia a interação (ex: Motorista, DevOps).

Lifeline / Participante: O componente executando a ação.

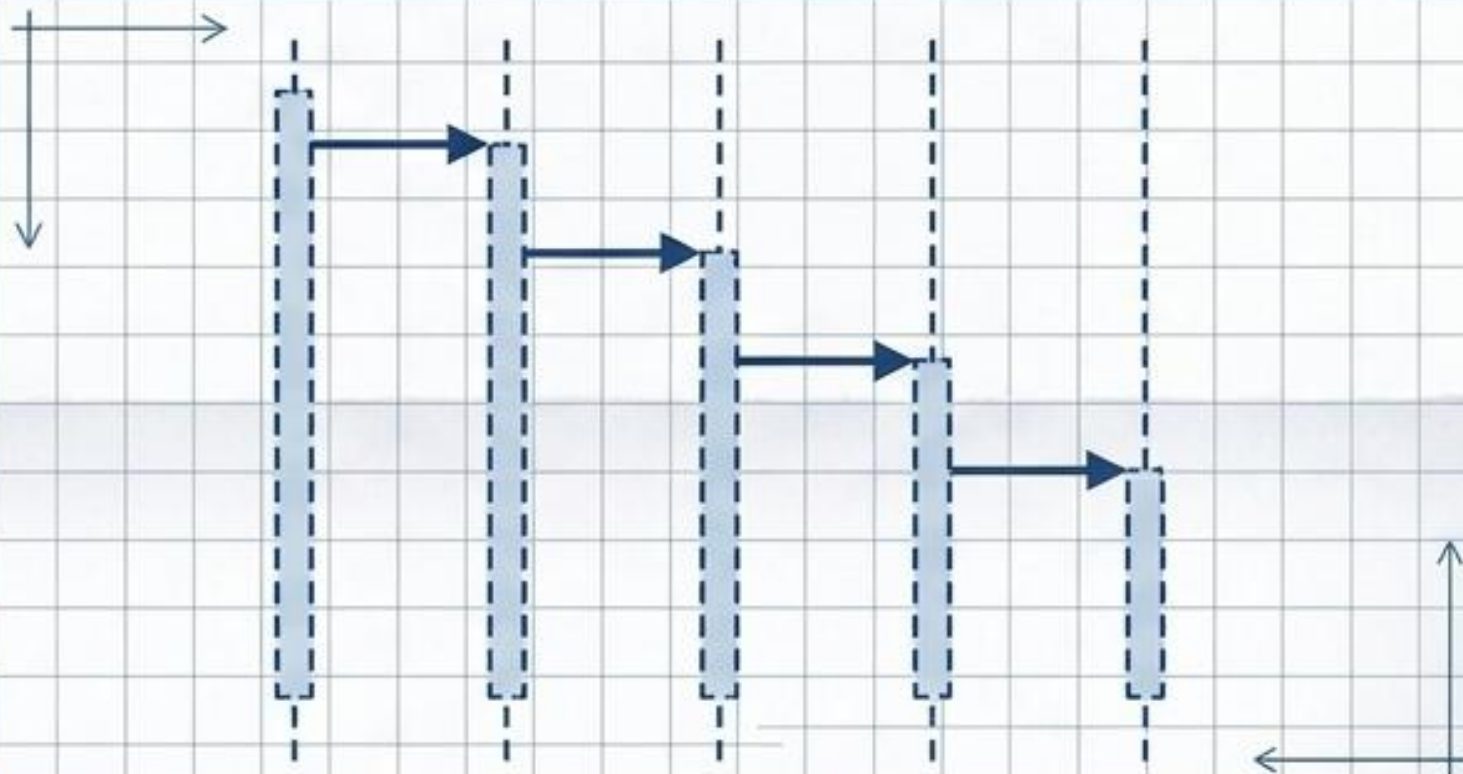
Mensagem: A comunicação exata e o protocolo (ex: POST /telemetry).

Ativação: O período de execução do componente.

Fragmento / Alt: Bifurcações, condições de sucesso/falha ou loops.

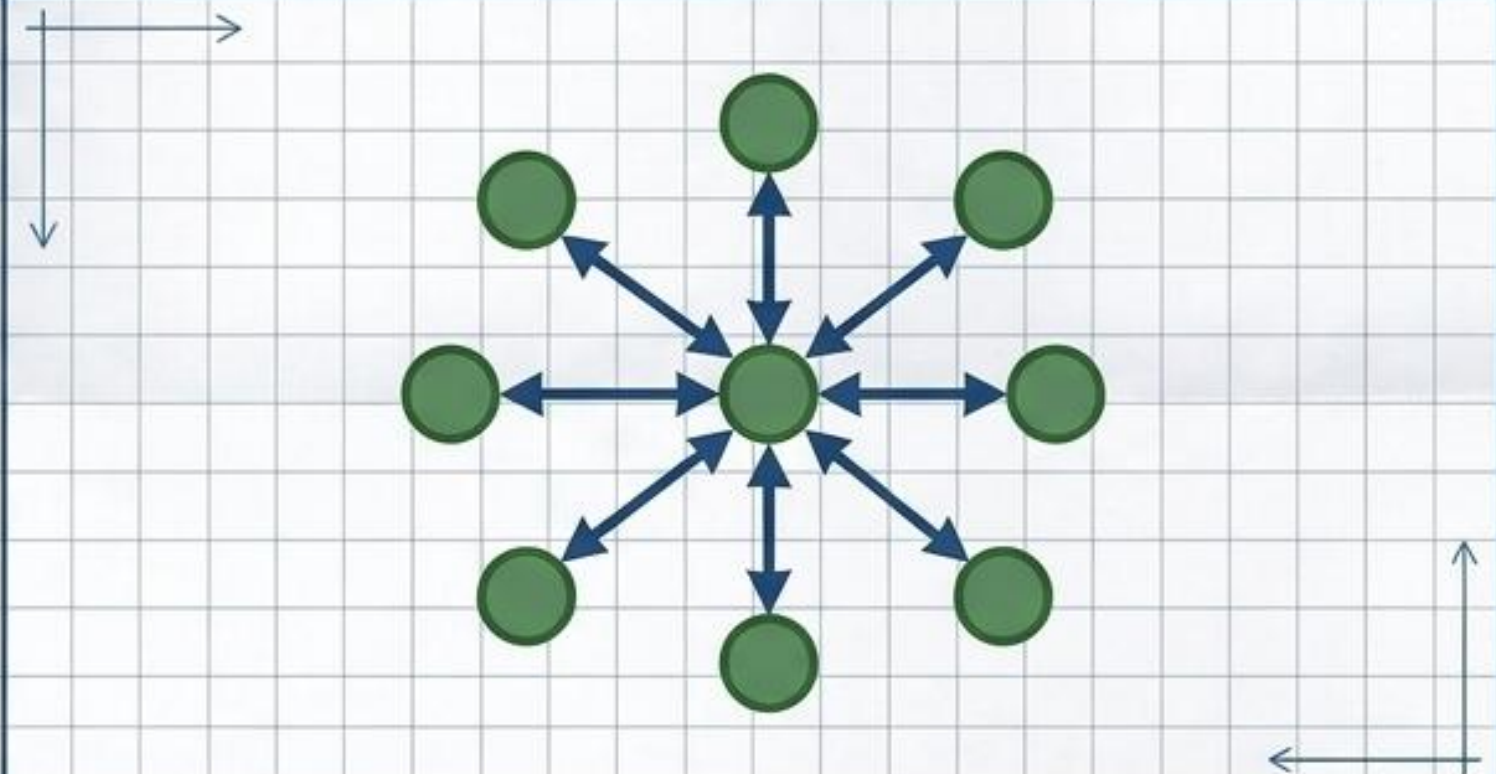
Duas Lentes para o Mesmo Sistema

Diagrama de Sequência



Ênfase:	Tempo (Ordem cronológica)
Layout:	Vertical
Uso ideal:	Rastrear fluxos detalhados e gargalos de latência.

Diagrama de Comunicação



Ênfase:	Estrutura (Conexões de rede)
Layout:	Horizontal (Grafo)
Uso ideal:	Visão panorâmica de interações e dependências.

Telemetria SwiftTrack: Os 4 Fluxos Vitais

FLUXO 1: Ingestão de Telemetria

[PRIORIDADE: CRÍTICA]

Ator: Motorista (IoT)

Modelo: Serverless

Meta: 5.000 TPS | < 80ms p95

FLUXO 2: Requisição Administrativa

[PRIORIDADE: CRÍTICA]

Ator: Operador

Modelo: IaaS Clássico

Meta: 1.000 TPS | < 200ms p95

FLUXO 3: Autenticação

[PRIORIDADE: ALTA]

Ator: Usuário

Modelo: Segurança / Identity

Meta: MFA | JWT 1h TTL

FLUXO 4: Deploy Automatizado

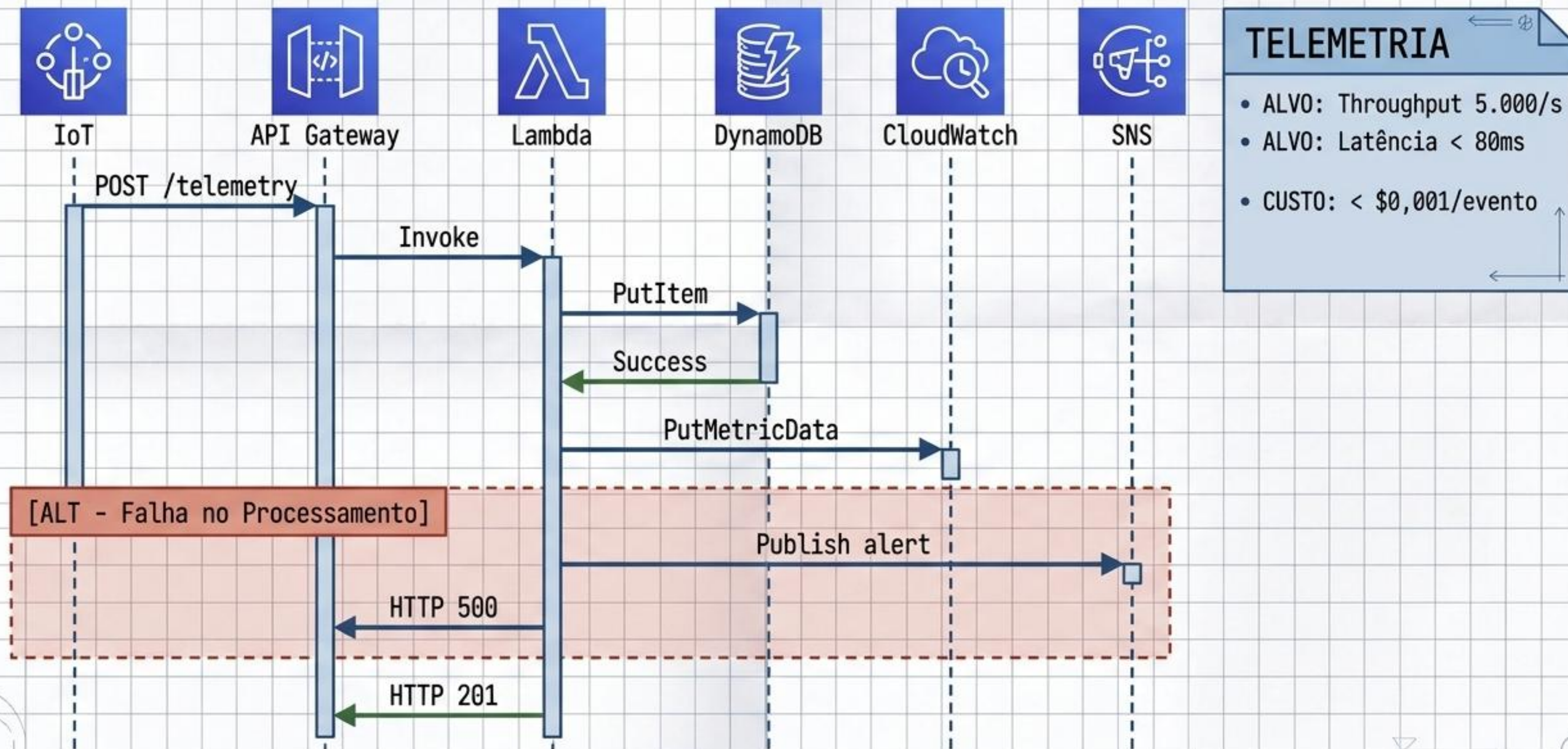
[PRIORIDADE: MÉDIA]

Ator: DevOps

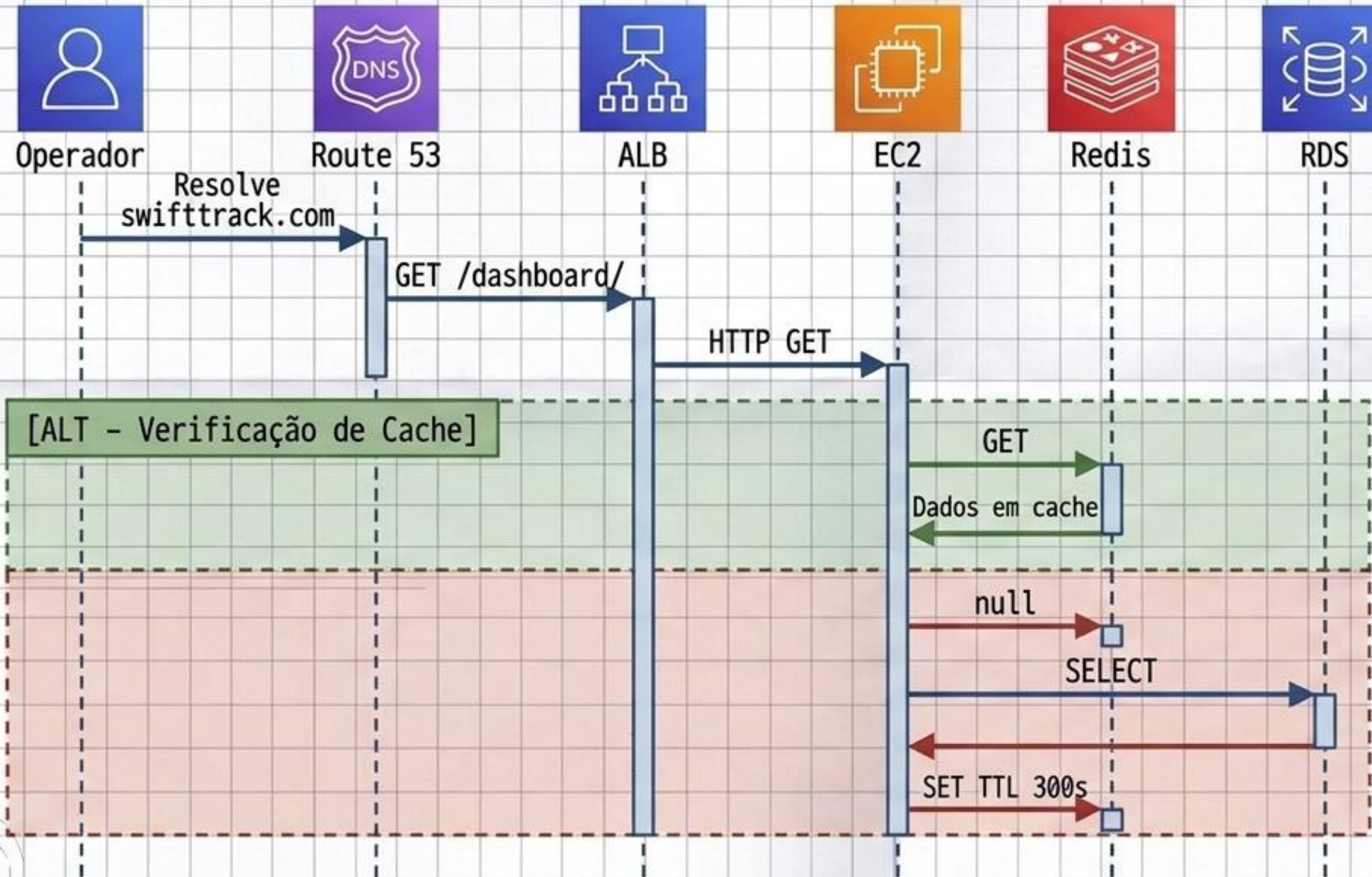
Modelo: CI/CD

Meta: Deploy < 10min | Zero Downtime

Raio-X 01: Ingestão de Telemetria (Serverless)



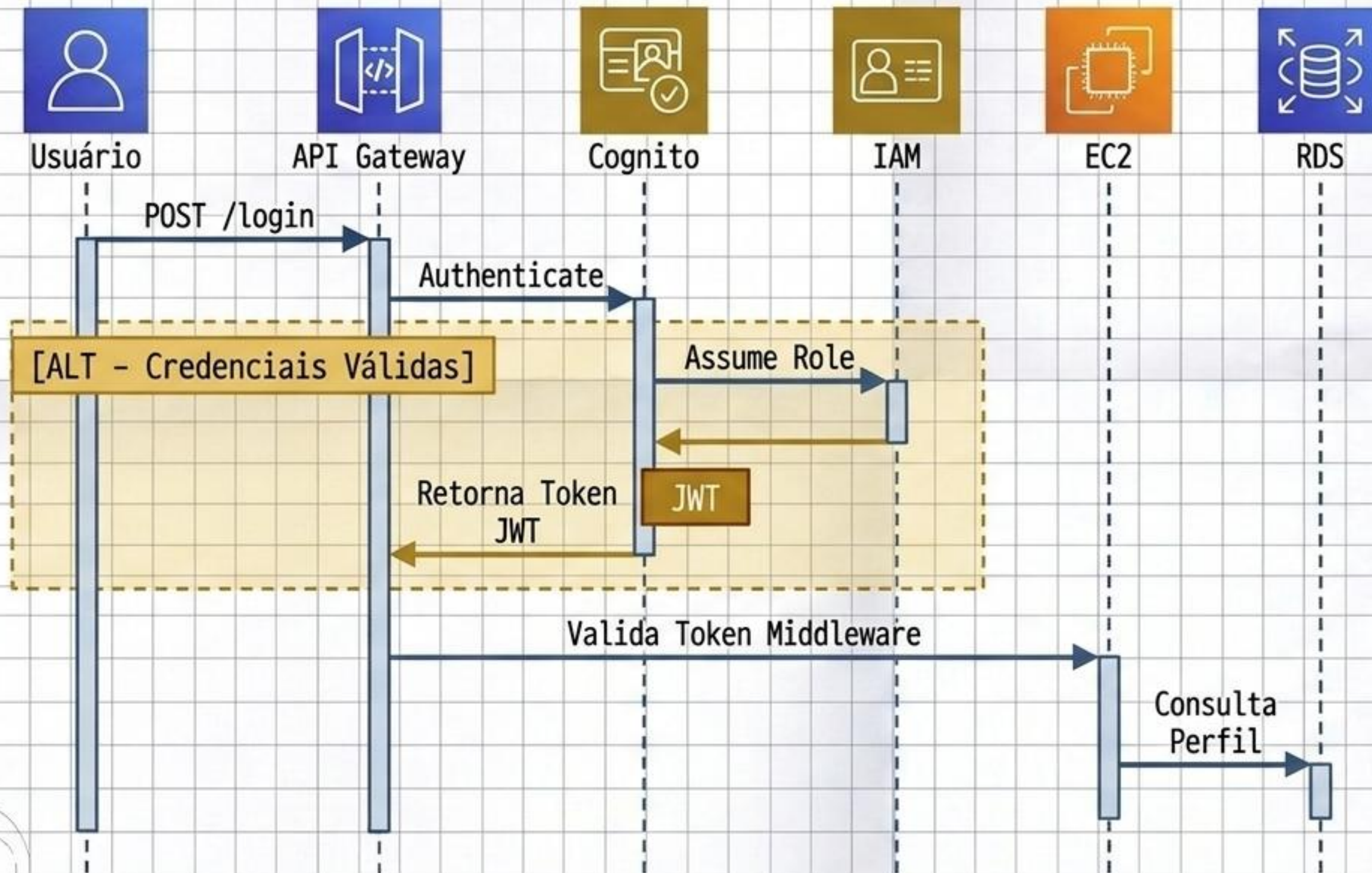
Raio-X 02: Requisição Administrativa (IaaS)



DIAGNÓSTICO

- DISPONIBILIDADE: 99,95%
- RISCO: Gargalo no RDS sem o Redis

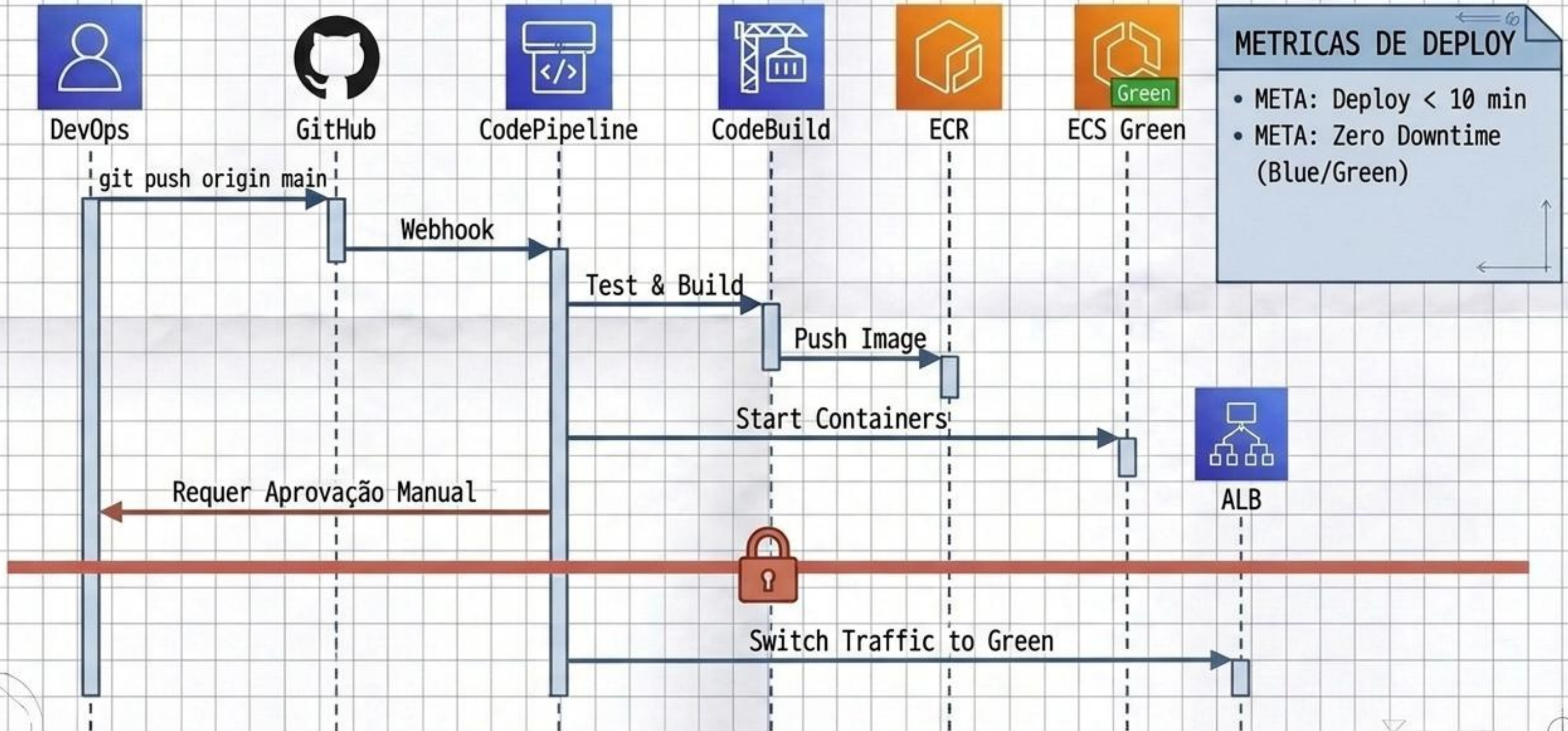
Raio-X 03: A Linha de Defesa (Auth)



REGRAS DE SEGURANÇA

- REGRA: MFA obrigatório para Admins
- REGRA: Vida útil do Token restrita (1 hora)

Raio-X 04: A Engrenagem CI/CD (Deploy Automatizado)

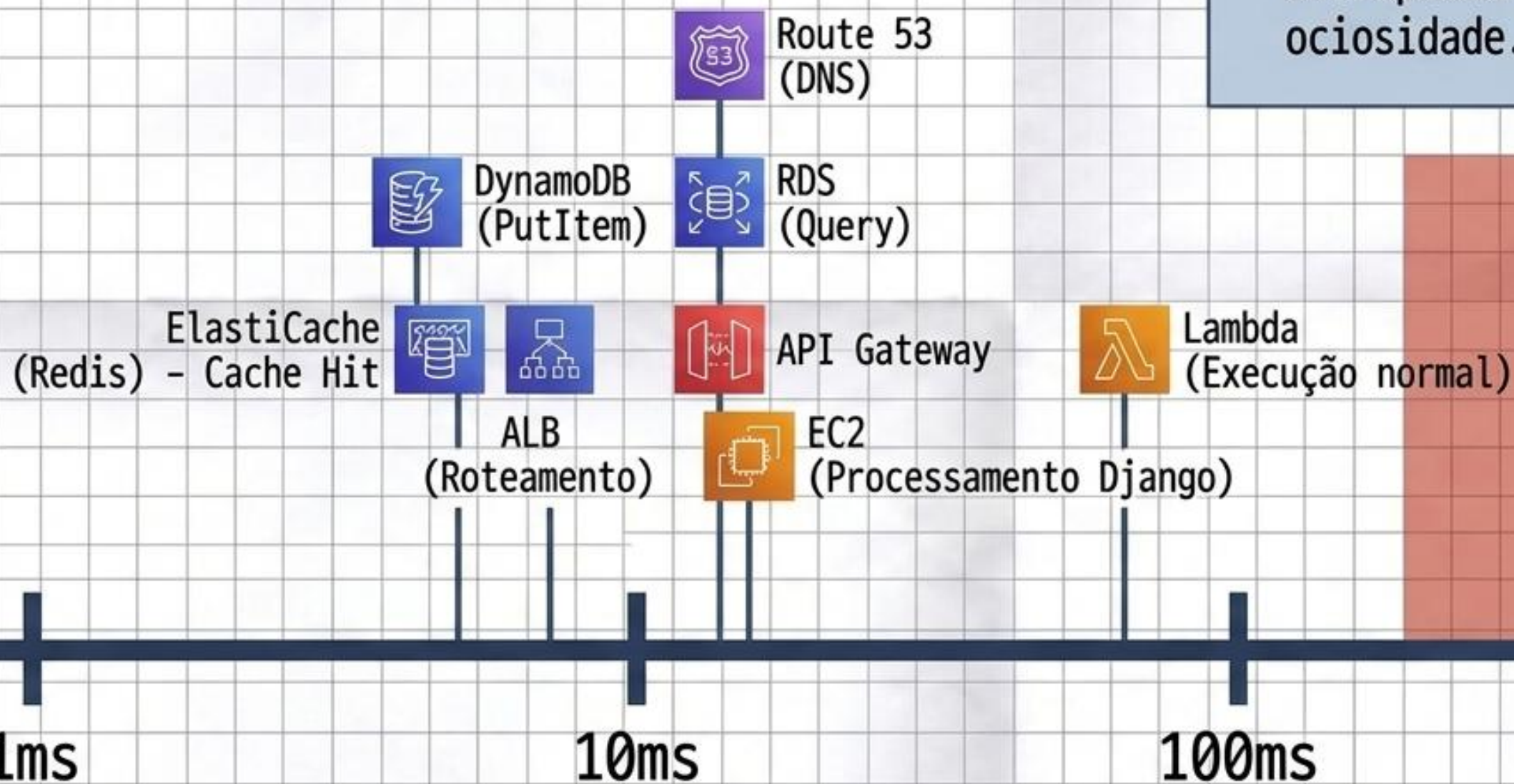


Matriz de Diagnóstico: Trade-offs Arquiteturais

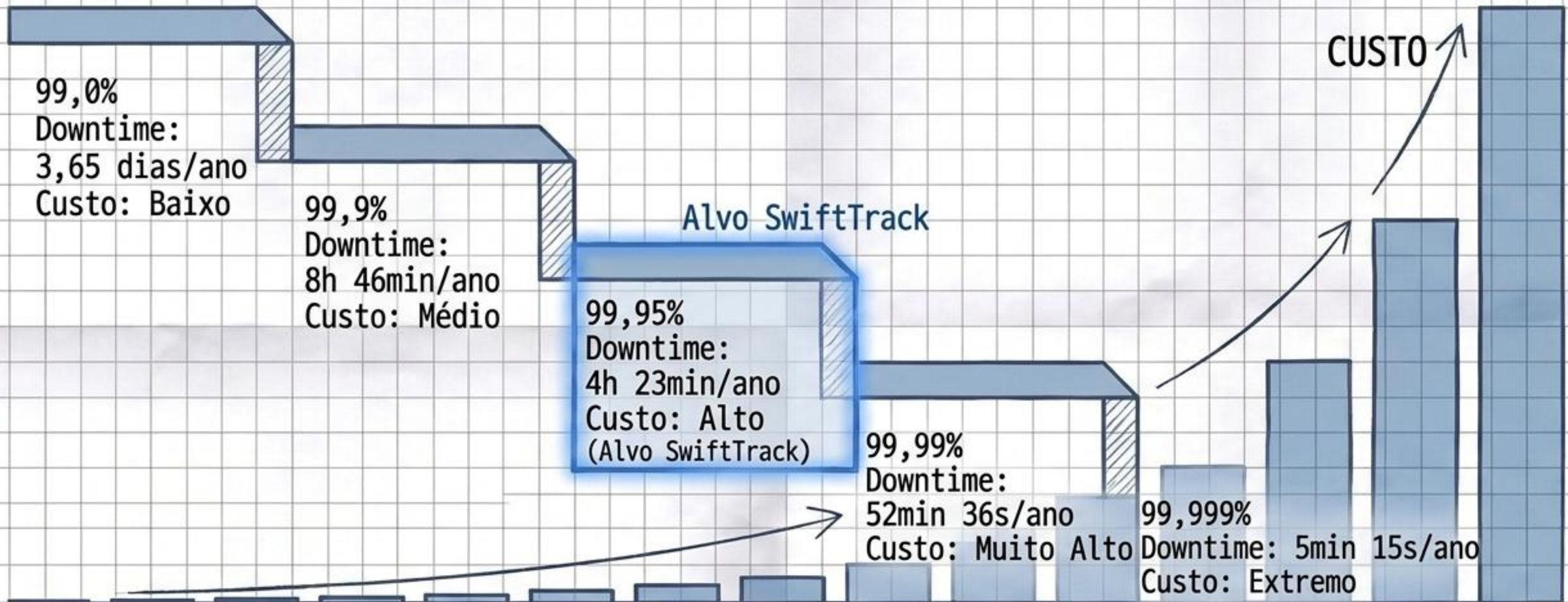
Arquitetura	Pró	Contra	Risco / Mitigação
FLUXO 01 [Serverless] 	• Escala infinita, custo por uso.	• Risco de Cold Start, limite de 15 min.	 Risco: Timeout.
FLUXO 02 [IaaS] 	• Controle total, latência previsível.	• Custo ocioso, gestão de instâncias.	 Mitigação: Gargalo via Redis.
FLUXO 03 [Auth] 	• Segurança delegada (Cognito), MFA nativo.	• Leve fricção no acesso.	 Risco: Dependência externa (Cognito down).
FLUXO 04 [CI/CD] 	• Rollback seguro (< 5 min), Zero downtime.	• Alta complexidade inicial de setup.	 Risco: Falha no switch do ALB.

Termômetro de Performance: Espectro de Latência

[!] ALERTA: Cold Start do Lambda pode ultrapassar 1.000ms em picos de ociosidade.



A Matemática da Disponibilidade (Os “Noves”)



Cada '9' adicional na arquitetura diminui o tempo de inatividade linearmente, mas aumenta a complexidade e o custo exponencialmente.

Console de Validação Arquitetural

- [✓] Atores e pontos de iniciação estão definidos?
- [✓] Mensagens seguem ordem cronológica estrita?
- [✓] Fragmentos (Alt/Loop) tratam exceções e falhas?
- [✓] Retornos estão explícitos para evitar deadlocks?
- [✓] Requisitos Não-Funcionais (NFRs) estão mapeados nos gargalos?
- [✓] Consistência garantida com Classes de Análise e Pacotes de Segurança?

STATUS: AVALIAÇÃO CONCLUÍDA

Próximos Passos (Semana 8)

A arquitetura está modelada e validada. O próximo passo é a compilação oficial do Documento de Arquitetura de Software (DAS), consolidando Visão, Requisitos, Casos de Uso e Modelos de Design.

LEITURA OBRIGATÓRIA

- > UML Distilled (Martin Fowler) – Capítulos de Sequência
- > AWS Well-Architected Framework – Operational Excellence Pillar

